Name:Naveen C H Roll No:231901033

Exp No: 7 ENDORSEMENT POLICY CONFIGURATION IN HYPERLEDGER FABRIC

AIM

To configure and implement endorsement policies in Hyperledger Fabric and verify how multiple organizations endorse transactions before they are committed to the blockchain ledger.

PROCEDURE:

Part 1: OR Endorsement Policy

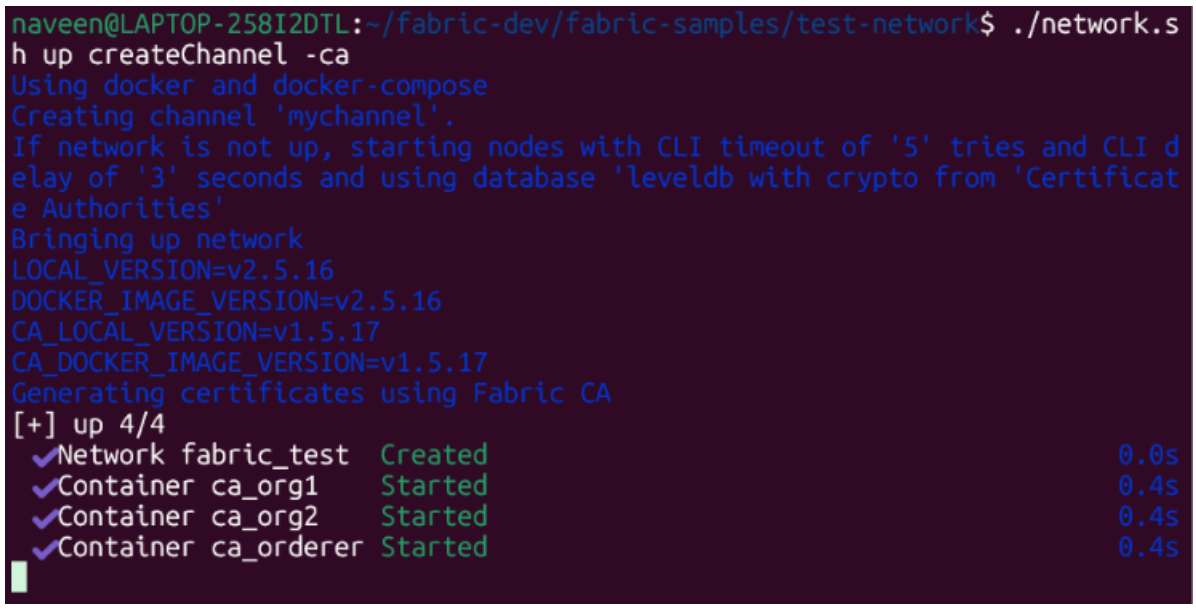
Step 1: Start Fresh

Navigate to the Fabric test network.

```
cd ~/fabric-dev/fabric-samples/test-network
```

```
./network.sh down
```

```
./network.sh up createChannel -ca
```



Step 2: Set Environment Variables

Load the Org1 environment.

```
source setOrg1.sh
```

Set the TLS certificate for Org1:

```
export
ORG1_TLS=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.o
rg1.example.com/tls/ca.crt
```

Set the TLS certificate for Org2:

```
export
ORG2_TLS=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.o
rg2.example.com/tls/ca.crt
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ source setO
rg1.sh
-bash: setOrg1.sh: No such file or directory
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ export ORG1
_TLS=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org
1.example.com/tls/ca.crt
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ export ORG2
_TLS=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org
2.example.com/tls/ca.crt
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ echo $ORG1_
TLS
echo $ORG2_TLS
/home/naveen/fabric-dev/fabric-samples/test-network/organizations/peerOrgani
zations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
/home/naveen/fabric-dev/fabric-samples/test-network/organizations/peerOrgani
zations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$
```

Step 3: Deploy Chaincode with OR Endorsement Policy

Deploy the Asset Transfer chaincode with a policy allowing either Org1 or Org2 to endorse transactions.

```
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-
basic/chaincode-javascript -ccep "OR('Org1MSP.peer','Org2MSP.peer')"
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ ./network.s
h deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode
-javascript -ccep "OR('Org1MSP.peer','Org2MSP.peer')"
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: OR('Org1MSP.peer','Org2MSP.peer')
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-bas
ic/chaincode-javascript --lang node --label basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
Using organization 1
+ peer lifecycle chaincode queryinstalled --output json
+ grep '^basic_1.0:1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7
f59b629$'
+ jq -r 'try (.installed_chaincodes[].package_id)'
+ test 1 -ne 0
+ peer lifecycle chaincode install basic.tar.gz
```

Step 4: Verify Chaincode Commitment

Query the committed chaincode definition.

```
peer lifecycle chaincode querycommitted -C mychannel --name basic
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ peer lifecy
cle chaincode querycommitted -C mychannel --name basic
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc
, Approvals: [Org1MSP: true, Org2MSP: true]
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$
```

Step 5: Initialize the Ledger

Since the policy is OR, endorsement from Org1 alone is sufficient.

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c
```

```
'{"function":"InitLedger","Args":[]}'
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ peer chainc
ode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c '{"function":"InitLedger","Args":[]}'
2026-09-01 18:04:21.400 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery
-> Chaincode invoke successful. result: status:200
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$
```

Step 6: Query All Assets

Retrieve all assets from the ledger.

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ peer chainc
ode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":
5,"docType":"asset"},{"AppraisedValue":400,"Color":"red","ID":"asset2","Owne
r":"Brad","Size":5,"docType":"asset"},{"AppraisedValue":500,"Color":"green",
"ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"},{"AppraisedValu
e":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"ass
et"},{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","
Size":15,"docType":"asset"},{"AppraisedValue":800,"Color":"white","ID":"asse
t6","Owner":"Michel","Size":15,"docType":"asset"}]
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$
```

Step 7: Stop the Network

```
./network.sh down
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ ./network.s
h down
Using docker and docker-compose
Stopping network
[+] down 10/10
 ✔Container ca_org2                        Removed                        3.6s
 ✔Container orderer.example.com            Removed                        0.5s
 ✔Container ca_org1                        Removed                        3.7s
 ✔Container ca_orderer                     Removed                        3.4s
 ✔Container peer0.org1.example.com         Removed                        0.9s
 ✔Container peer0.org2.example.com         Removed                        1.1s
 ✔Volume compose_peer0.org1.example.com    Removed                        0.0s
 ✔Network fabric_test                      Removed                        0.2s
 ✔Volume compose_peer0.org2.example.com    Removed                        0.0s
 ✔Volume compose_orderer.example.com       Removed                        0.0s
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volum
e
Error response from daemon: get docker_peer0.org2.example.com: no such volum
```

Part 2: AND Endorsement Policy

Step 8: Restart the Network

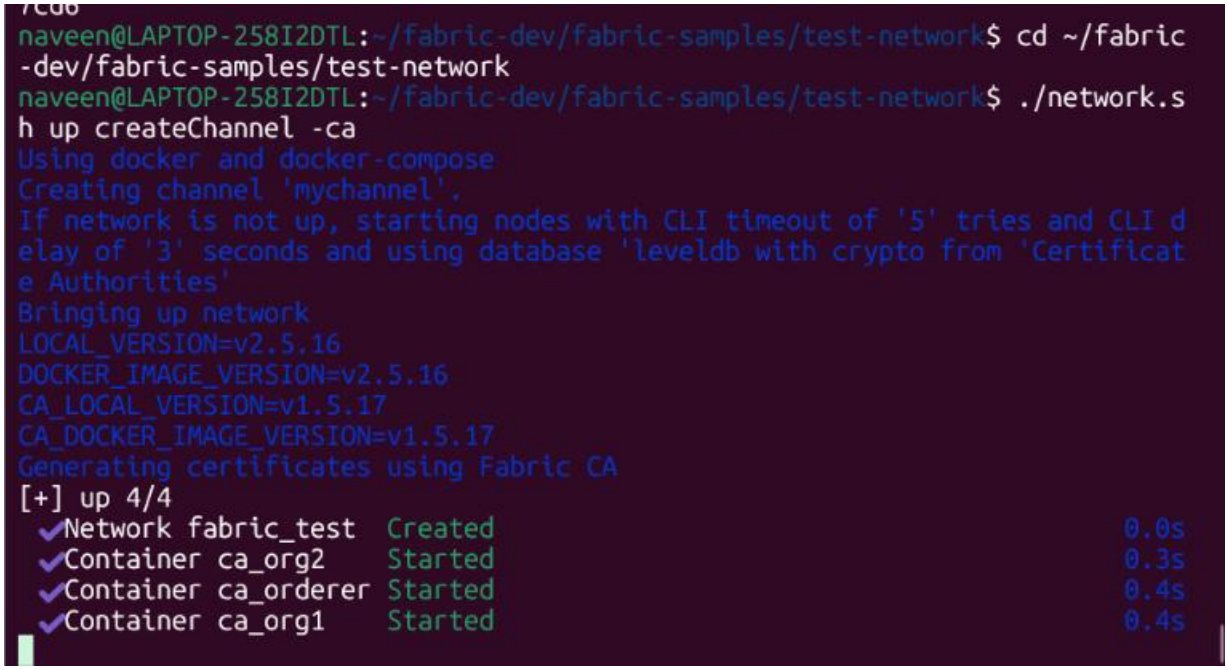
Start the Fabric network again.

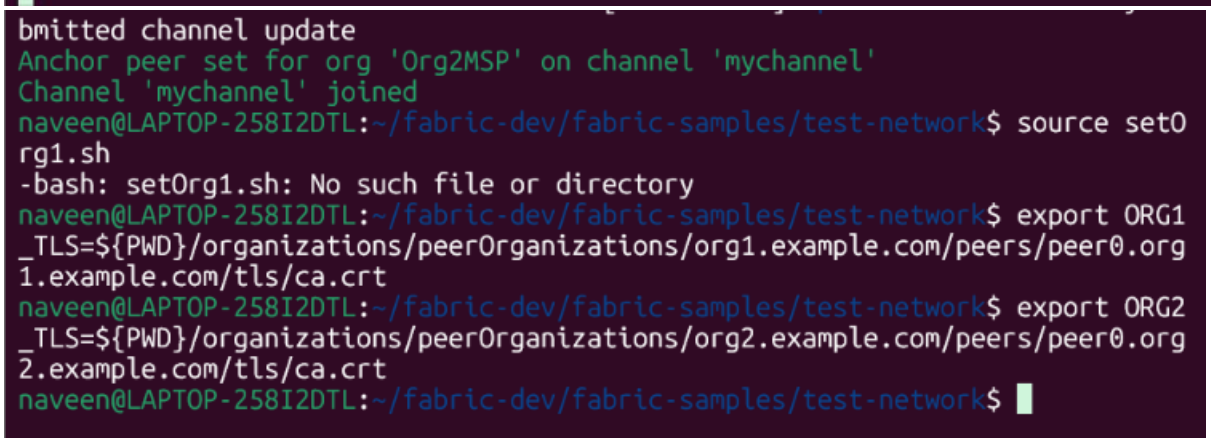
```
./network.sh up createChannel -ca
```

Load the Org1 environment.

```
source setOrg1.sh
```

Set the TLS certificates using the same ORG1_TLS and ORG2_TLS variables shown above.





Step 9: Deploy Chaincode with AND Endorsement Policy

Deploy the chaincode with a policy requiring both organizations to endorse transactions.

```
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-
basic/chaincode-javascript -ccep "AND('Org1MSP.peer','Org2MSP.peer')"
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ ./network.s
h deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode
-javascript -ccep "AND('Org1MSP.peer','Org2MSP.peer')"
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: AND('Org1MSP.peer','Org2MSP.peer')
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
```

Step 10: Initialize the Ledger

Since the AND policy requires both organizations, the transaction must be sent to both Org1 and Org2 peers.

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic --
peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_TLS --peerAddresses
localhost:9051 --tlsRootCertFiles $ORG2_TLS -c
'{"function":"InitLedger","Args":[]}'
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ peer chainc
ode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $ORG1_TLS \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $ORG2_TLS \
-c '{"function":"InitLedger","Args":[]}'
2026-09-01 18:08:39.001 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery
-> Chaincode invoke successful. result: status:200
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$
```

Name:Naveen C H Roll No:231901033

Step 11: Create Asset25

Create the asset using endorsements from both organizations.

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic --
peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_TLS --peerAddresses
localhost:9051 --tlsRootCertFiles $ORG2_TLS -c
'{"function":"CreateAsset","Args":["asset25","Blue","10","Arun","1200"]}'
```

Wait for the transaction to complete successfully before querying the asset.

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ peer chainc
ode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $ORG1_TLS \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $ORG2_TLS \
-c '{"function":"CreateAsset","Args":["asset25","Blue","10","Arun","1200"]}'
2026-09-01 18:09:19.290 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery
-> Chaincode invoke successful. result: status:200 payload:"{\"ID\":\"asset2
5\",\"Color\":\"Blue\",\"Size\":10,\"Owner\":\"Arun\",\"AppraisedValue\":120
0}"
```

Step 12: Verify the Asset

Query asset25 from the ledger.

```
peer chaincode query -C mychannel -n basic -c
'{"Args":["ReadAsset","asset25"]}'
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ peer chainc
ode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset25"]}'
{"AppraisedValue":1200,"Color":"Blue","ID":"asset25","Owner":"Arun","Size":1
0}
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$
```

Step 13: Stop the Network

```
./network.sh down
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ ./network.s
h down
Using docker and docker-compose
Stopping network
[+] down 10/10
 ✔Container peer0.org1.example.com      Removed                          0.9s
 ✔Container ca_orderer                  Removed                          3.6s
 ✔Container peer0.org2.example.com      Removed                          1.1s
 ✔Container orderer.example.com         Removed                          0.5s
 ✔Container ca_org2                     Removed                          3.5s
 ✔Container ca_org1                     Removed                          3.4s
 ✔Volume compose_peer0.org1.example.com Removed                          0.0s
 ✔Volume compose_peer0.org2.example.com Removed                          0.0s
 ✔Network fabric_test                   Removed                          0.2s
 ✔Volume compose_orderer.example.com    Removed                          0.0s
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volum
e
Error response from daemon: get docker_peer0.org2.example.com: no such volum
e
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2
```

RESULT

The endorsement policies were successfully configured and tested in the Hyperledger Fabric network. The OR policy demonstrated that either organization can endorse a transaction, while the AND policy demonstrated that endorsements from both organizations are required. The transactions were successfully validated and committed according to the configured policies, demonstrating secure and trusted transaction processing in a permissioned blockchain network.